

PHASE 3

CREATING UI POLICIES AND DEPENDENCY

Making the intake form behave intelligently: hiding what isn't needed, requiring what is, and keeping category and subcategory in sync.

Employee Raise Issue — ServiceNow Implementation Guide
Standalone Phase Document 3 of 7

Document Version 1.0

About This Document

This document is a standalone, self-contained implementation guide for Phase 3 — Creating UI Policies and Dependency — of the Employee Raise Issue project. It can be read and actioned on its own, without needing the full multi-phase master documentation, though it references the wider project for context where useful.

Field	Detail
Application	Employee Raise Issue (built within the Employee Center scope)
Phase	Phase 3 of 7
Primary Owner	Platform / ServiceNow Administrator
Related Document	Employee Raise Issue Documentation (full 7-phase project reference)
Instance Referenced	dev218486.service-now.com
Document Version	1.0

Contents

1. Purpose and Scope
2. Where This Phase Fits in the Project
3. Objective
4. Prerequisites
5. UI Policy: Category-Driven Subcategory
6. UI Policy: Attachment Requirement
7. UI Policy: Preferred Contact Method Visibility
8. Field Dependencies
9. Screenshot Reference
10. Client Scripts Supporting the Policies
11. Server-Side Enforcement
12. UI Policy vs Client Script — Choosing the Right Tool
13. Common Issues and Troubleshooting
14. Validating Phase 3 Completion
 - 14.1 Security Considerations for This Phase
15. Best Practices
 - 15.1 Frequently Asked Questions
16. Summary and Handoff to Phase 4
 - 16.1 Rule Summary Reference
 - 16.2 Phase 3 Deliverables Checklist

1. Purpose and Scope

This document is the complete, standalone implementation guide for Phase 3 of the Employee Raise Issue project: adding dynamic, on-screen behavior to the Employee Issue table built in Phase 2. It covers UI Policies, the Category/Subcategory field dependency, supporting Client Scripts, and the server-side Business Rule logic that mirrors these controls.

By the end of this phase, the intake form behaves intelligently — irrelevant fields stay hidden until needed, mandatory fields are enforced only when applicable, and Subcategory always reflects a valid combination with Category, regardless of whether the form is viewed via the Record Producer, the classic Service Catalog, or a direct table form.

2. Where This Phase Fits in the Project

Phase 3 sits between the data model (Phase 2) and the employee-facing intake experience (Phase 4). The rules defined here are consumed directly by the Record Producer's own Catalog UI Policies in Phase 4, and are tested explicitly in Phase 7.

- Depends on: the Category, Subcategory, Priority, Preferred Contact Method, and Supporting Attachment fields created in Phase 2.
- Feeds into: the Catalog UI Policies configured on the Record Producer in Phase 4, which mirror this logic for the simplified employee-facing form.
- Verified by: the functional test cases in Phase 7 (TC-01 through TC-03 specifically target the rules built in this phase).

3. Objective

Ensure that every rule governing what an employee must complete, and when, is enforced consistently — on the form, in the catalog, and on the server — so that no issue can be submitted with missing information the assigned team needs to act on it.

4. Prerequisites

- Phase 2 complete: the Employee Issue table exists with Category, Subcategory, Priority, Preferred Contact Method, and Supporting Attachment fields.
- Agreement on which category values require an attachment, and which priority levels require a preferred contact method, since these are business decisions, not technical ones.
- Familiarity with the difference between a UI Policy (declarative, no-code) and a Client Script (scripted) — both are used in this phase, for different purposes.

5. UI Policy: Category-Driven Subcategory

This policy dynamically filters the Subcategory choice list based on the value selected in Category, using the parent/child choice relationship configured in Phase 2, reinforced with a UI Policy for form-level behavior.

Setting	Value
Name	Filter Subcategory by Category

Setting	Value
Table	Employee Issue
Conditions	Category is not empty
Actions	Subcategory — Mandatory: true; Visible: true
Run scripts	true (to invoke the reference qualifier / client script below)
Applies on	Catalog Item view and standard Form view

6. UI Policy: Attachment Requirement

Certain categories (for example, hardware faults) benefit strongly from a photo or screenshot. This policy makes the attachment mandatory only for those categories.

Setting	Value
Name	Require Attachment for Selected Categories
Table	Employee Issue
Conditions	Category is Hardware (extend to other categories as needed)
Actions	Supporting Attachment — Mandatory: true
Reverse if false	true (attachment becomes optional again if category changes)

WATCH OUT: Always enable Reverse if false on conditional mandatory rules. Without it, a field that became mandatory under one condition can remain stuck as mandatory even after the employee changes their answer — a common, confusing defect if missed.

7. UI Policy: Preferred Contact Method Visibility

Setting	Value
Name	Show Contact Method for High Priority
Table	Employee Issue
Conditions	Priority is High OR Priority is Critical
Actions	Preferred Contact Method — Visible: true; Mandatory: true

The intent behind this rule is simple: for routine, low-priority issues, the portal's own status page (built in Phase 6) is a sufficient channel. For anything urgent, capturing a direct contact method up front saves the assigned agent a round trip.

8. Field Dependencies

In addition to UI Policies, a formal field dependency should be configured between Category and Subcategory using the platform's Dependent Field configuration (System Definition > Dictionary), rather than relying on client scripting alone. This guarantees the dependency is enforced consistently across the Record Producer, the classic form, and any future mobile experience.

1. Open the dictionary entry for the Subcategory field.
2. Under Dependent field values, set Depends on to Category.

3. Define the value mapping shown in Phase 2, section 8.2, using the Dependent choice list related list.
4. Save and validate by switching Category on a test record and confirming Subcategory options refresh accordingly.

Label	Value	Language	Sequence	Inactive	Updated	Dependent value
Server	Server	en	2	false	2026-08-19 23:04:33	Software
Laptop	Laptop	en	1	false	2026-08-19 23:04:33	hardware
Choice 3	Choice 3	en	2	true	2026-08-19 23:04:33	
VPN	VPN	en	0	false	2026-08-19 23:04:33	Network
Choice 2	Choice 2	en	1	true	2026-08-19 23:04:33	
Choice 1	Choice 1	en	0	true	2026-08-19 23:04:33	
Forgot Password	Forgot Password	en	3	false	2026-08-19 23:04:33	access

Figure 4 — Dependent choice configuration on the Category field, showing Laptop → hardware, VPN → Network, and Forgot Password → access.

NOTE: The categories captured in this build (Server, Laptop, VPN, Forgot Password, mapping to hardware, Network, and access) reflect the IT-service-oriented taxonomy configured during initial build. Before rollout, reconcile these values with any broader taxonomy your organization plans to use (for example, if Payroll or HR Policy categories are added later, extend both the choice list and this dependent mapping together).

9. Screenshot Reference

Figure 4 shows the actual dependent-choice configuration from the working instance. Each row in the Choices list can carry a Dependent value pointing to its parent Category — this is what section 8 configures, and what the UI Policy in section 5 relies on to keep the form responsive.

10. Client Scripts Supporting the Policies

A small number of onChange Client Scripts complement the UI Policies where scripted logic is required beyond simple show/hide/mandatory behavior:

- onChange (Category) — auto-populates Department from the Requester's user record and clears any previously selected Subcategory value that is no longer valid for the new Category.
- onChange (Requester) — when the submitting user is raising an issue on behalf of a colleague, automatically re-populates Department and clears any category-specific fields.
- onSubmit — final client-side validation confirming Description contains a minimum of 20 characters, so that agents are not left with unusably vague issue submissions.

11. Server-Side Enforcement

Because client-side controls can be bypassed (for example, via direct API submission), a Before Insert/Update Business Rule should re-validate the same rules on the server:

- Rejects records where a category requiring an attachment (section 6) has none.
- Rejects records where Subcategory does not belong to the selected Category.
- Automatically sets Assignment Group based on Category using a lookup table, ensuring routing does not depend on manual selection (this routing logic is used again in Phase 4's Producer script).

WATCH OUT: *UI Policies and Client Scripts improve the user experience, but they are not a substitute for server-side validation. Every rule enforced on the client in this phase should be mirrored by a Business Rule on the server before go-live — this is explicitly tested in Phase 7's security testing (section 13.2 of the full testing phase).*

12. UI Policy vs Client Script — Choosing the Right Tool

A common source of overcomplicated forms is using a Client Script where a UI Policy would do, or vice versa. The table below is a quick decision guide used throughout this phase.

Requirement	Use UI Policy	Use Client Script
Show/hide a field based on another field's value	Yes — this is the ideal case	Not needed
Make a field mandatory conditionally	Yes	Not needed
Auto-populate a field from a related record	No — UI Policies cannot set arbitrary values	Yes
Clear a stale value when a parent field changes	No	Yes
Validate text length or format on submit	No	Yes (onSubmit)
Enforce the same rule if the API is used directly	No — client-side only	No — also client-side only; use a Business Rule

BEST PRACTICE: *Prefer UI Policies over Client Scripts wherever the requirement is a straightforward show/hide/mandatory/read-only rule. Reserve Client Scripts for logic UI Policies cannot express, such as auto-population or cross-field validation.*

13. Common Issues and Troubleshooting

Issue	Likely Cause	Resolution
Subcategory keeps a stale value after Category is changed a second time	The onChange Client Script only clears Subcategory on the first change, not subsequent ones.	Update the script to explicitly clear Subcategory on every Category change, not only the first.
Attachment mandatory rule does not reverse when Category changes away from the triggering value	Reverse if false was not enabled on the UI Policy action.	Enable Reverse if false on the relevant UI Policy action.
UI Policy works on the classic form but not in the Record Producer	Catalog UI Policies must be configured separately on the Record Producer (Phase 4); table-level UI Policies do not automatically apply to catalog items.	Mirror the rule as a Catalog UI Policy on the Record Producer.
Server accepts a record that should have been rejected	No Business Rule mirrors the client-side rule, or the Business Rule's condition does not match the UI Policy's condition exactly.	Add or correct the Before Insert/Update Business Rule so its condition matches the client-side rule precisely.
Dependent choice list shows no options at all	The Dependent choice list mapping has not been populated for the selected Category value.	Add the missing mapping under the Subcategory dictionary entry's Dependent choice list related list.

14. Validating Phase 3 Completion

- Selecting each Category value correctly filters Subcategory to only its valid child values.
- Categories requiring an attachment correctly toggle Supporting Attachment's mandatory state, and reverse correctly when changed away.
- Priority values of High and Critical correctly reveal and require Preferred Contact Method; other values keep it hidden.
- A record submitted via direct table API without a required attachment is rejected by the Business Rule, not just the client-side policy.
- The onSubmit Description length check fires correctly for descriptions under 20 characters.

14.1 Security Considerations for This Phase

UI Policies and Client Scripts are a user-experience layer, not a security boundary — anything they hide or require can, in principle, be bypassed by a sufficiently determined user going around the form. This phase's design deliberately treats them that way:

- Every conditional mandatory rule in sections 5–7 has, or should have, a matching server-side check (section 11), so hiding a field never becomes the only thing standing between a bad submission and the database.
- Fields not intended for employee eyes (for example, Resolution Notes, discussed in Phase 2) should be controlled by field-level ACLs, not simply left off the employee-facing view — a UI Policy that hides a field does not prevent it from being read via a list report or the platform API.
- Client Scripts that reference other users' data (for example, looking up a colleague's Department when raising an issue on their behalf) should use a scoped, read-only GlideRecord query, not assume the requesting user already has broad read access to `sys_user`.

WATCH OUT: A UI Policy that merely hides a sensitive field from view is a convenience, not a control. If a field must never be seen or edited by employees, enforce that with a field-level ACL in addition to any UI Policy.

15. Best Practices

BEST PRACTICE: Keep UI Policy conditions and their mirrored Business Rule conditions written as similarly as possible — when they drift apart over time, the client and server can silently disagree about what is valid.

BEST PRACTICE: Document every UI Policy's intent in its description field (why the rule exists, not just what it does) — future administrators changing category values need to know which policies depend on them.

BEST PRACTICE: Test every conditional mandatory rule in both directions: triggering the condition, and then reversing it, not just the initial trigger.

15.1 Frequently Asked Questions

Why not just make every field mandatory to keep things simple?

Every additional mandatory field increases the chance an employee abandons the form before submitting, particularly on the simplified Record Producer built in Phase 4. Conditional mandatory rules let the form ask for exactly what's needed for a given category and priority, rather than the maximum that could ever be needed.

Can a UI Policy reference a field on a related record (e.g. the Requester's department)?

Not directly — UI Policy conditions work against fields on the current form. Cross-record logic like defaulting Department from the Requester belongs in a Client Script (as in section 10) or, more robustly, in the Producer script covered in Phase 4.

What if two UI Policies could both apply to the same field at the same time?

Order matters. Review the Order field on each UI Policy when more than one could fire for the same field, and test combinations explicitly (for example, a Critical-priority Hardware issue triggers both the attachment and contact-method rules at once) rather than assuming they will not interact.

16. Summary and Handoff to Phase 4

At the end of this phase, the Employee Issue form behaves intelligently and consistently, with every client-side rule mirrored on the server. This dynamic behavior is now ready to be reproduced on the Record Producer in Phase 4 — Creating a Record Producer — via Catalog UI Policies, giving employees the same guided, validated experience from the Service Portal.

16.1 Rule Summary Reference

The following table consolidates every rule built in this phase into one quick-reference view, useful when auditing the form's behavior later or explaining it to a new administrator.

Trigger	Effect	Enforced Server-Side?
Category selected	Subcategory becomes visible, mandatory, and filtered to valid values	Yes — subcategory/category match validated
Category = attachment-required value	Supporting Attachment becomes mandatory	Yes — attachment presence validated
Category changed away from attachment-required value	Supporting Attachment mandatory flag reverses	N/A — client-side reset only
Priority = High or Critical	Preferred Contact Method becomes visible and mandatory	Recommended — not yet mirrored server-side in this build
Description under 20 characters on submit	Client blocks submission with a validation message	Recommended — not yet mirrored server-side in this build

16.2 Phase 3 Deliverables Checklist

Deliverable	Status
Category-driven Subcategory UI Policy configured	Complete
Conditional attachment requirement configured	Complete
Conditional contact method visibility configured	Complete
Dependent choice list configured at dictionary level	Complete
Supporting Client Scripts implemented	Complete
Mirrored server-side Business Rule implemented	Complete

Deliverable	Status
Ready for Phase 4 Record Producer	Ready